



Universidad
Nacional
de Loja

UNIVERSIDAD NACIONAL DE LOJA

**Facultad de la Energía, las Industrias y los Recursos
Naturales no Renovables**

GRUPO 5

Uso de Wokwi y Microchip Studio en microcontroladores AVR

Práctica Nro. 001

INTEGRANTES:

Martina Alejandra Maldonado Machuca
Victor Fernando Roa Carrión
Emilio José Galvez Merchan
Wilson Patricio Palma Samaniego

Ciclo: Quinto

Asignatura: Sistemas Digitales

Docente: Ing. Ramiro Rivera
LOJA-ECUADOR

Ciudad Universitaria "Guillermo Falconí Espinosa" Casilla letra "S"

Teléfono: 2547 – 252 Ext. 101: 2547-200

Ciudad Unive

1. DESARROLLO DE LA PRÁCTICA.....	3
1.1 Parte A. Reconocimiento del entorno y del microcontrolador.....	3
1.1.1 A1. Creación del proyecto en Wokwi.....	3
1.1.2 A2. Identificación del ATmega328P, LED integrado y D13.....	3
1.1.3 A3. Código 1 — Blink con la API de Arduino.....	4
1.2 Parte B. El mismo comportamiento, ahora con registros.....	4
1.2.1 B1. Código 2 — Blink escribiendo registros.....	4
1.2.2 B2. Encendido y apagado con OR y AND.....	5
1.2.3 B3. Conmutación mediante PINB.....	6
1.2.4 B4. Asignación directa en PORTB.....	7
1.3 Parte C. Circuito y laboratorio de bits.....	7
1.3.1 C1. Construcción del circuito.....	7
1.3.2 C2. Código 3 — Laboratorio de bits.....	8
1.3.3 C3. Operaciones binarias.....	10
1.3.4 C4. Modos de funcionamiento.....	11
1.3.4.1 Modo 0 — Contador binario de 4 bits.....	11
1.3.4.2 Modo 1 — Rotación circular.....	12
1.3.4.3 Modo 2 — Alternancia mediante XOR.....	12
1.3.5 C5. Identificación de bits en PORTB.....	13
1.3.6 C6. Asignación directa en escribirLeds().....	14
1.3.7 C7. Tabla 4 — Configuración de registros.....	15
1.4 Parte D. Costo de la API frente al acceso a registros.....	15
1.4.1 D1. Código 4 — Medición de tiempo de escritura.....	15
1.4.2 D2. Mediciones realizadas.....	16
1.5 Parte E. Del C al set de instrucciones AVR con Microchip Studio.....	17
1.5.1 E1. Creación del proyecto en Microchip Studio.....	17
1.5.2 E2. Código 5 y compilación.....	18
1.5.3 E3. Archivo .iss y Tabla 5.....	19
1.5.4 E4. Selección del simulador.....	20
1.5.5 E5. Vista I/O.....	20
1.5.6 E6. Contador de ciclos.....	21
2. Enlaces Públicos.....	21
2.1 Wokwi.....	21
2.2 Github.....	22
3. RESPUESTAS A LAS PREGUNTAS DE CONTROL.....	22
4. CONCLUSIONES.....	24
5. DECLARACIÓN DE USO DE IA GENERATIVA.....	24
6. REFERENCIAS.....	25

1. DESARROLLO DE LA PRÁCTICA

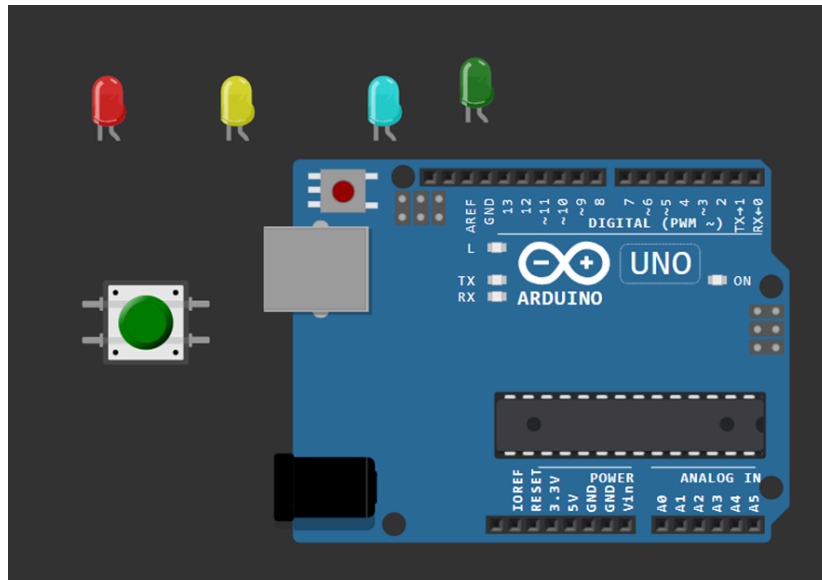
1.1 Parte A. Reconocimiento del entorno y del microcontrolador

1.1.1 A1. Creación del proyecto en Wokwi

- Enlace del Proyecto

<https://wokwi.com/projects/476790060388494337>

- Evidencia del proyecto



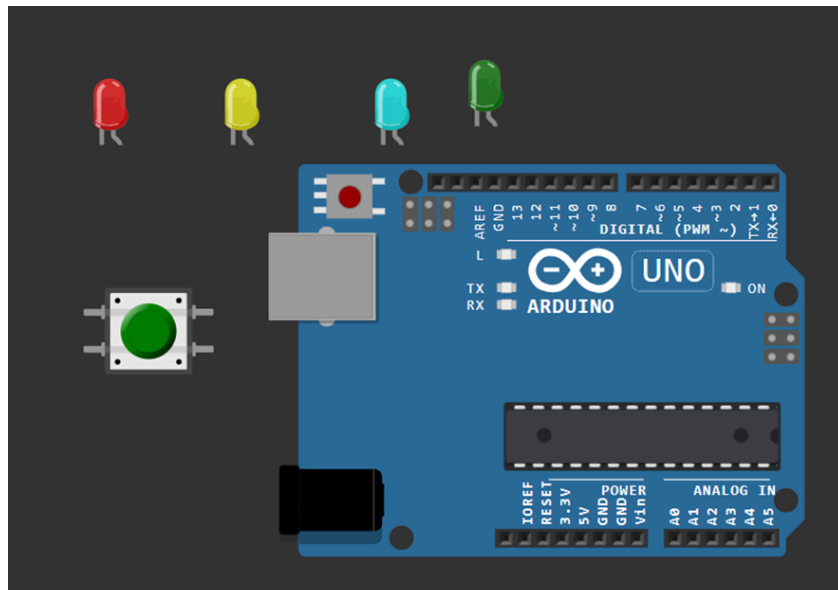
1.1.2 A2. Identificación del ATmega328P, LED integrado L y D13

En la placa simulada se identificó el microcontrolador ATmega328P, el LED integrado L y el pin D13. La correspondencia es:

D13 → PB5

Por tanto, D13 pertenece al puerto B, bit 5, y se controla mediante DDRB, PORTB y PINB.

- Evidencia:



1.1.3 A3. Código 1 — Blink con la API de Arduino

Se utilizó como línea base el siguiente código:

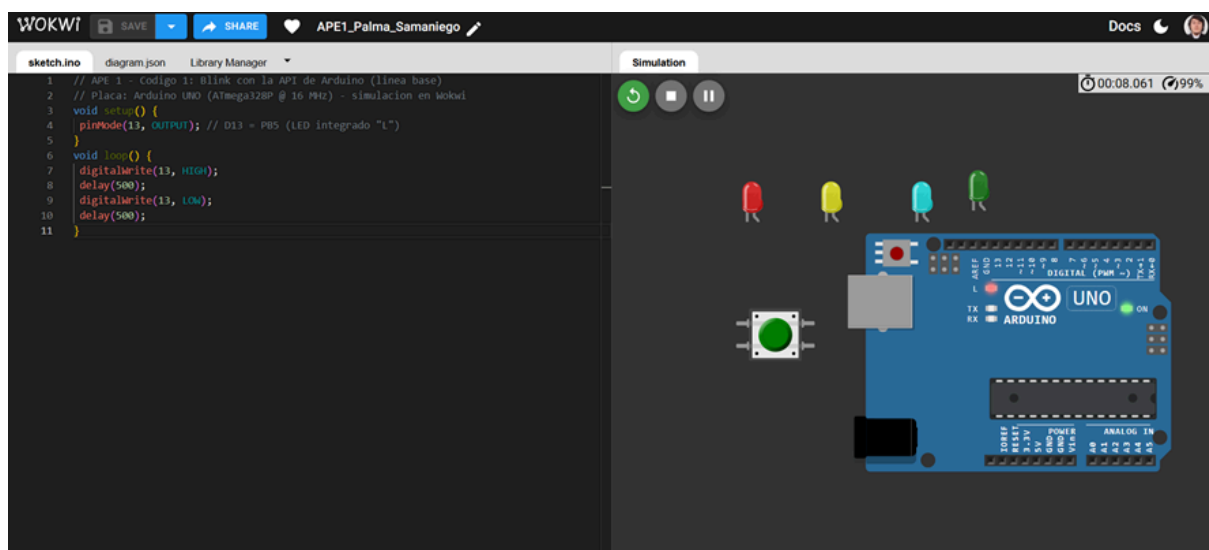
```
// APE 1 - Codigo 1: Blink con la API de Arduino (línea base)
// Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulacion en Wokwi

void setup() {
  pinMode(13, OUTPUT); // D13 = PB5 (LED integrado "L")
}

void loop() {
  digitalWrite(13, HIGH);
  delay(500);
  digitalWrite(13, LOW);
  delay(500);
}
```

Resultado obtenido: el LED integrado L parpadeó correctamente, con aproximadamente 500 ms encendido y 500 ms apagado.

Evidencia:



1.2 Parte B. El mismo comportamiento, ahora con registros

1.2.1 B1. Código 2 — Blink escribiendo registros

Se reemplazó el Código 1 por el siguiente:

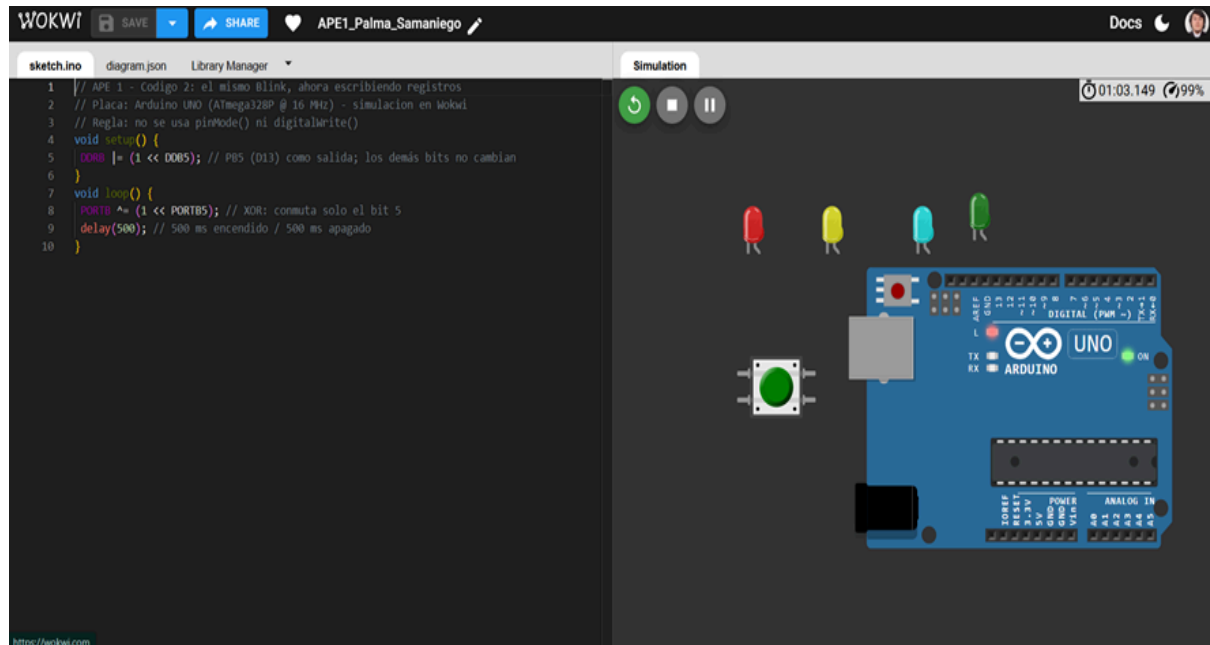
```
// APE 1 - Codigo 2: el mismo Blink, ahora escribiendo registros
// Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulacion en Wokwi
// Regla: no se usa pinMode() ni digitalWrite()

void setup() {
  DDRB |= (1 << DDB5); // PB5 (D13) como salida; los demás bits no cambian
}
```

```
void loop() {
  PORTB ^= (1 << PORTB5); // XOR: conmuta solo el bit 5
  delay(500); // 500 ms encendido / 500 ms apagado
}
```

Resultado obtenido: el comportamiento del LED L fue igual al observado con el Código 1.

Evidencia:



1.2.2 B2. Encendido y apagado con OR y AND

Se reemplazó la conmutación por:

```
void loop() {
  PORTB |= (1 << PORTB5);
  delay(500);

  PORTB &= ~(1 << PORTB5);
  delay(500);
}
```

Resultado obtenido: el LED L continuó parpadeando correctamente.

La operación utilizada para encender el bit fue:

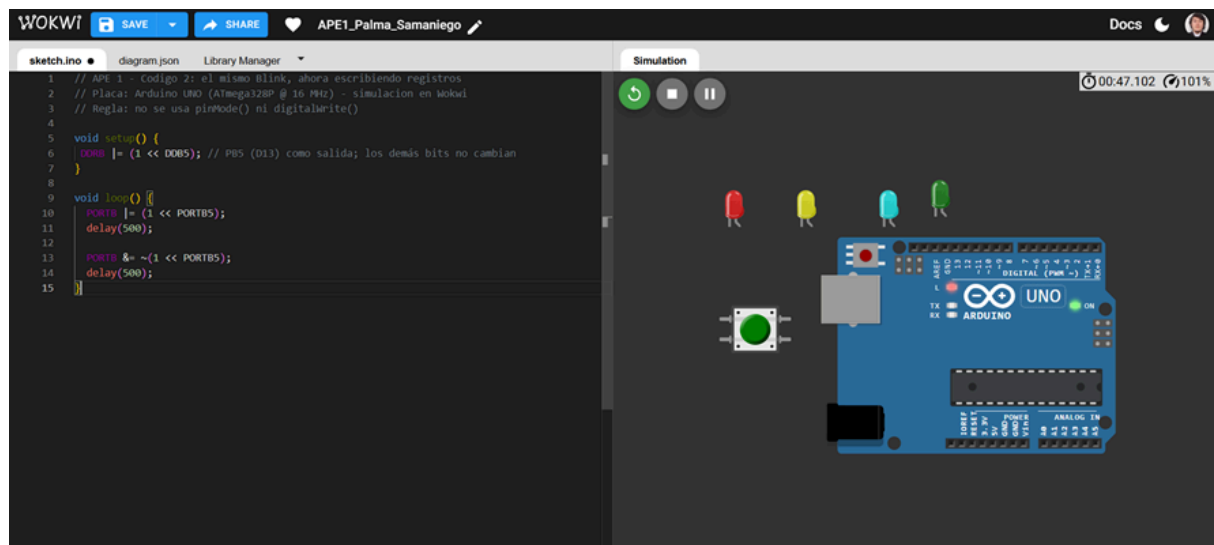
`PORTB |= (1 << PORTB5);`

La operación utilizada para apagar el bit fue:

`PORTB &= ~(1 << PORTB5);`

Estas operaciones permiten modificar el bit 5 sin sobrescribir los demás bits de PORTB.

Evidencia:



1.2.3 B3. Conmutación mediante PINB

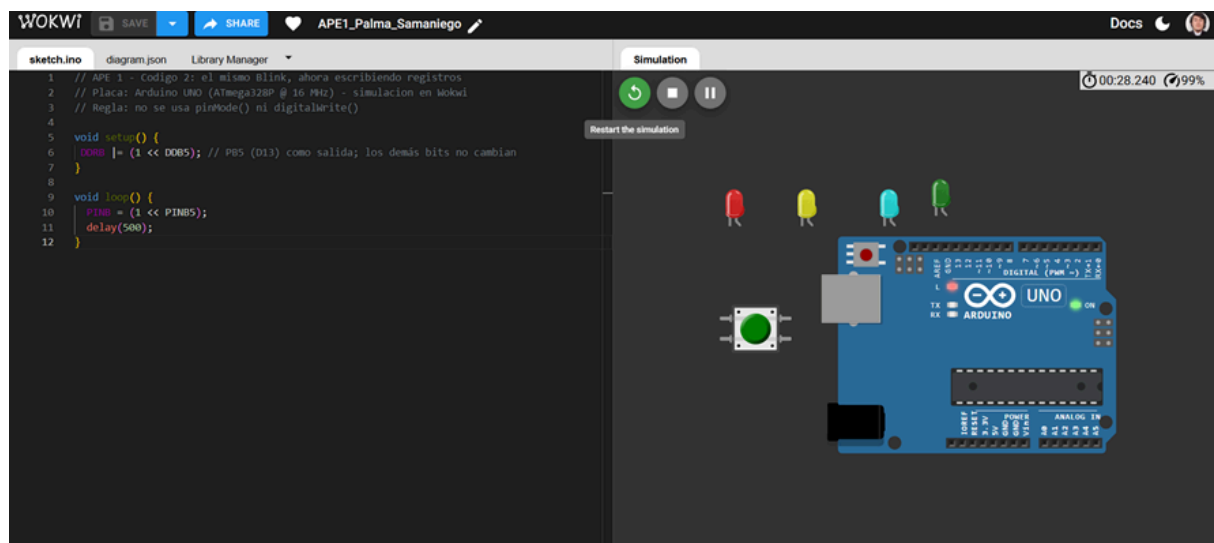
Se probó:

```
void loop() {
  PINB = (1 << PINB5);
  delay(500);
}
```

Resultado obtenido: el LED L continuó conmutando.

La característica utilizada es que, en el ATmega328P, **escribir un 1 en un bit de PINx conmuta el bit correspondiente de PORTx**. En este caso, escribir un 1 en PINB5 conmuta PB5.

Evidencia:



1.2.4 B4. Asignación directa en PORTB

Se probó:

```
void loop() {  
  PORTB = (1 << PORTB5);  
  delay(500);  
  
  PORTB = 0;  
  delay(500);  
}
```

La asignación directa reemplaza el contenido completo de PORTB. Por ello, los otros bits del registro quedan determinados por el valor escrito y pueden cambiar aunque no se quisiera modificarlos. Si PB0, PB1, PB2 y PB3 estuvieran conectados a otros LEDs, una asignación como:

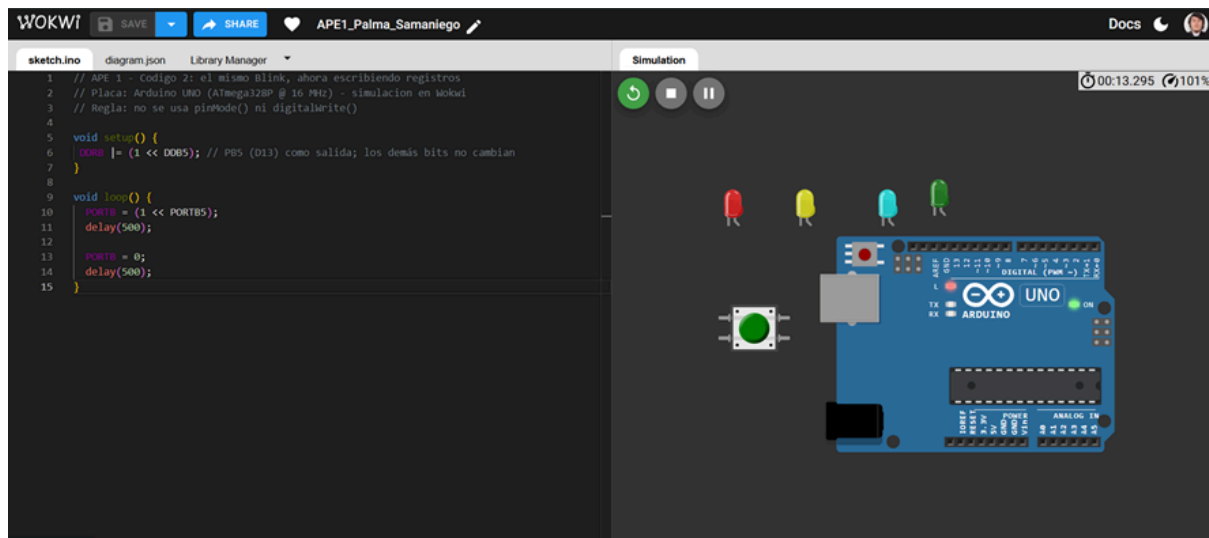
```
PORTB = (1 << PORTB5);
```

haría que PB0, PB1, PB2 y PB3 pasaran a 0. En consecuencia, los LEDs controlados por esos bits cambiarían de estado. Esto es diferente de:

```
PORTB |= (1 << PORTB5);
```

porque la operación OR conserva los demás bits del registro

.Evidencia:



1.3 Parte C. Circuito y laboratorio de bits

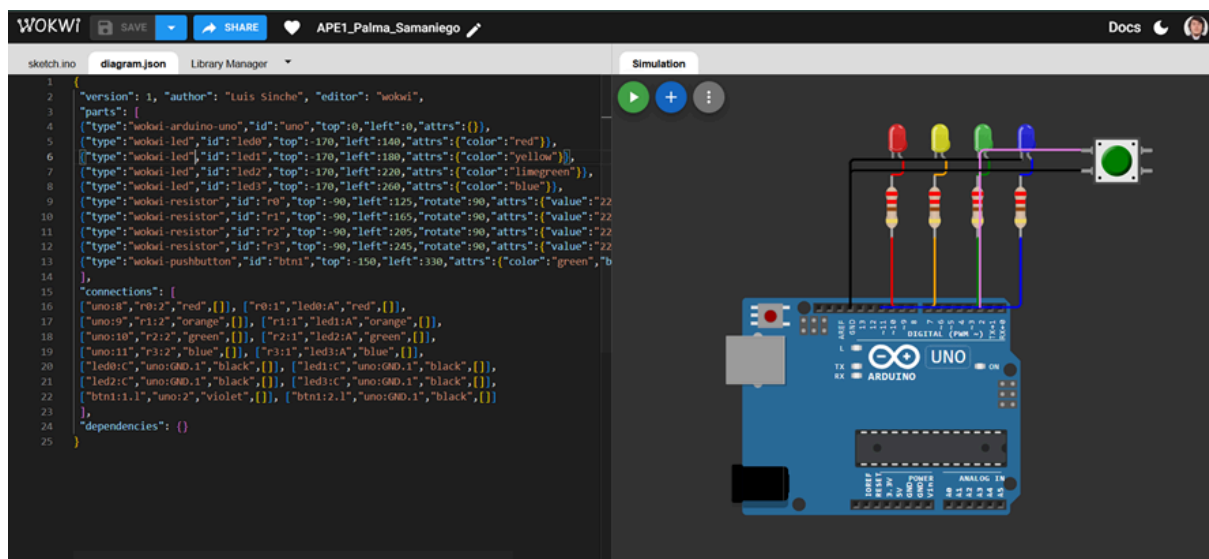
1.3.1 C1. Construcción del circuito

Se construyó en Wokwi el circuito indicado en la guía:

Componente	Pin del componente	Pin Arduino UNO	Puerto / bit	Configuración
LED0 (rojo) + R0 220 Ω	Ánodo vía R0	D8	PB0	Salida
LED1 (amarillo) + R1 220 Ω	Ánodo vía R1	D9	PB1	Salida
LED2 (verde) + R2 220 Ω	Ánodo vía R2	D10	PB2	Salida
LED3 (azul) + R3 220 Ω	Ánodo vía R3	D11	PB3	Salida
LED0 ... LED3	Cátodo	GND	—	—
Pulsador	1.I	D2	PD2	Entrada con pull-up interna
Pulsador	2.I	GND	—	—
LED integrado "L"	—	D13	PB5	Salida (latido)

El pulsador se conectó a GND para utilizar la resistencia pull-up interna. Por ello, en reposo se lee 1 y al presionar se lee 0

Evidencia del circuito:



1.3.2 C2. Código 3 — Laboratorio de bits

Se cargó el siguiente código en Wokwi y se utilizó el Monitor Serie a 9600 baudios:

/*

APE 1 - Codigo 3: Laboratorio de bits en los puertos B y D
 Placa : Arduino UNO (ATmega328P @ 16 MHz) - simulacion en Wokwi
 Libs : ninguna (avr/io.h llega incluido por Arduino.h)
 Pines : PB0..PB3 (D8..D11) LEDs | PD2 (D2) pulsador a GND | PB5 (D13) LED L
 Regla : NO se usa pinMode(), digitalWrite() ni digitalRead()

*/

```
const uint8_t MASK_LEDS = 0x0F;
const uint8_t NUM_MODOS = 3;
const uint8_t PATRON_INICIAL[NUM_MODOS] = {0b0000, 0b0001, 0b0101};
```



```

const unsigned long PERIODO_MS = 300;

uint8_t modo = 0;
uint8_t patron = PATRON_INICIAL[0];
uint8_t btnAnterior = 1;
unsigned long tAnterior = 0;

void imprimirBin8(const char *etiqueta, uint8_t v) {
    Serial.print(etiqueta);

    for (int8_t i = 7; i >= 0; i--) {
        Serial.print((v >> i) & 1);
    }

    Serial.println();
}

void escribirLeds(uint8_t valor) {
    PORTB = (PORTB & ~MASK_LEDS) | (valor & MASK_LEDS);
}

void demoOperaciones() {
    uint8_t a = 0b11001100;
    uint8_t b = 0b10101010;

    imprimirBin8("a = ", a);
    imprimirBin8("b = ", b);
    imprimirBin8("a & b = ", a & b);
    imprimirBin8("a | b = ", a | b);
    imprimirBin8("a ^ b = ", a ^ b);
    imprimirBin8("~a = ", (uint8_t)~a);
    imprimirBin8("a << 1 = ", (uint8_t)(a << 1));
    imprimirBin8("a >> 2 = ", a >> 2);

    Serial.print("~a sin cast, en BIN: ");
    Serial.println(~a, BIN);
}

void setup() {
    DDRB |= MASK_LEDS | (1 << DDB5);

    DDRD &= ~(1 << DDD2);

    PORTD |= (1 << PORTD2);

    Serial.begin(9600);

    Serial.println(F("APE 1 - Laboratorio de bits"));

    demoOperaciones();

    escribirLeds(patron);
}

void loop() {
    uint8_t btn = (PIND >> PIND2) & 1;

```

```

if (btnAnterior == 1 && btn == 0) {
    modo = (modo + 1) % NUM_MODOS;

    patron = PATRON_INICIAL[modo];

    Serial.print(F("Modo: "));
    Serial.println(modo);
}

btnAnterior = btn;

if (millis() - tAnterior >= PERIODO_MS) {

    tAnterior = millis();

    switch (modo) {

        case 0:
            patron = (patron + 1) & MASK_LEDS;
            break;

        case 1:
            patron = ((patron << 1) | (patron >> 3)) & MASK_LEDS;
            break;

        case 2:
            patron ^= MASK_LEDS;
            break;
    }

    escribirLeds(patron);

    PINB = (1 << PINB5);

    imprimirBin8("PORTB = ", PORTB);
}
}

```

1.3.3 C3. Operaciones binarias

Para:

a = 11001100
b = 10101010

se obtuvieron:

a & b = 10001000
a | b = 11101110
a ^ b = 01100110
~a = 00110011
a << 1 = 10011000
a >> 2 = 00110011

```
Serial.println(~a, BIN);
```

```
(uint8_t)~a
```

[illegible]

WOKWT

SAVE

SHARE

APE1_Palma_Samaniego

Docs

sketch.ino

diagram.json

Library Manager

```

1
2 APE 1 -Codigo 3: Laboratorio de bits en los puertos B y D
3 Placa : Arduino Uno (ATmega328P @ 16 MHz) - simulacion en wokwi
4 Libs : ninguna (avr/io.h llega incluido por Arduino.h)
5 Pines : PB0..PB3 (0x..011) LEDs | PD2 (02) pulsador a GND | PBS (013) LED L
6 Regla : NO se usa pinMode(), digitalWrite() ni digitalRead()
7
8
9 const uint8_t MASK_LEDS = 0x0F;
10 const uint8_t NUM_MODOS = 3;
11 const uint8_t PATRON_INICIAL[NUM_MODOS] = {0b0000, 0b0001, 0b0101};
12 const unsigned long PERIODO_MS = 300;
13
14 uint8_t modo = 0;
15 uint8_t patron = PATRON_INICIAL[0];
16 uint8_t btnAnterior = 1;
17 unsigned long tAnterior = 0;
18
19 void imprimirBin(const char *etiqueta, uint8_t v) {
20   Serial.print(etiqueta);
21
22   for (int8_t i = 7; i >= 0; i--) {
23     Serial.print((v >> i) & 1);
24   }
25
26   Serial.println();
27 }
28
29 void escribirBcd(uint8_t valor) {
30   PORTB = (PORTB & ~MASK_LEDS) | (valor & MASK_LEDS);
31 }
32

```

Simulation

▶

■

⏸

04:10.571

978%

```

APE 1 - Laboratorio de bits
a = 11001100
b = 10101010
a & b = 10001000
a | b = 11101110
a ^ b = 01100110
~a = 00110011
a << 1 = 10011000
a >> 2 = 00110011
~a sin cast, en BIN: 111111111111111111111111111100110011

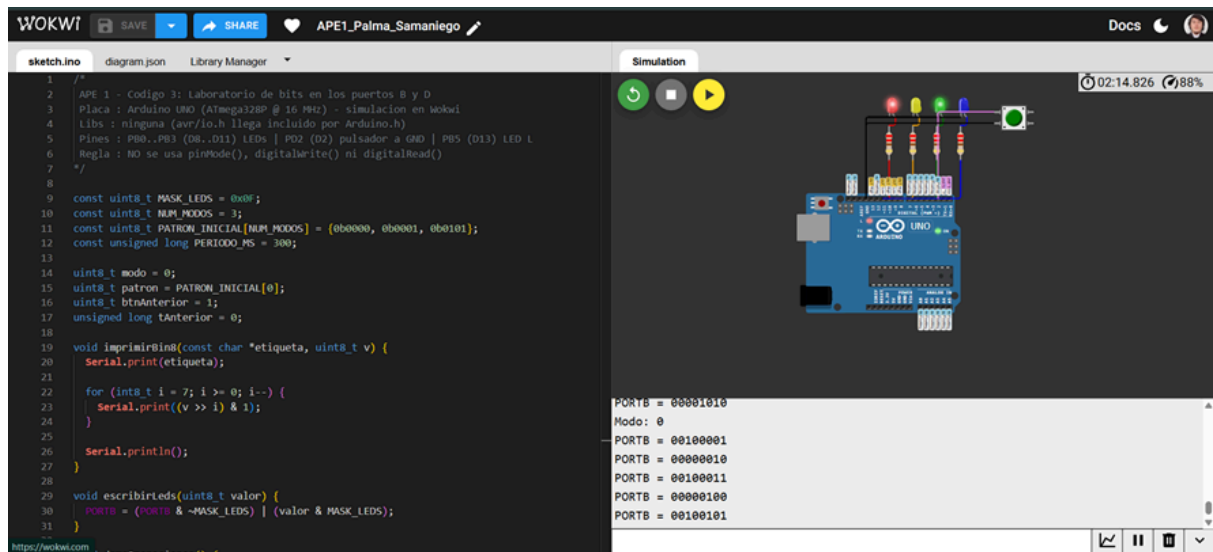
```

1.3.4.1 Modo 0 — Contador binario de 4 bits

```
patron = (patron + 1) & MASK LEDs;
```

Los cuatro LEDs actúan como contadores.

Evidencia:



1.3.4.2 Modo 1 — Rotación circular

La operación es:

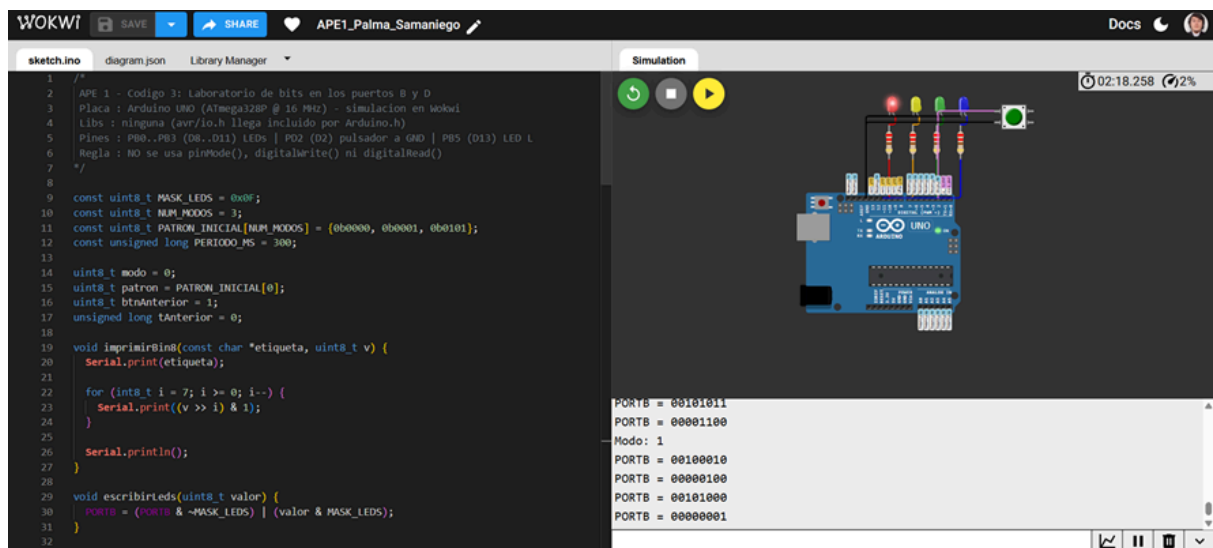
$$\text{patron} = ((\text{patron} \ll 1) | (\text{patron} \gg 3)) \& \text{MASK_LEDS};$$

La expresión desplaza el patrón hacia la izquierda y reincorpora por el extremo derecho el bit que salió por el extremo izquierdo. De esta manera se produce una rotación circular entre los cuatro LEDs.

Observación obtenida:

Se activa cada LED de manera secuencial.

Evidencia:



1.3.4.3 Modo 2 — Alternancia mediante XOR

La operación es:

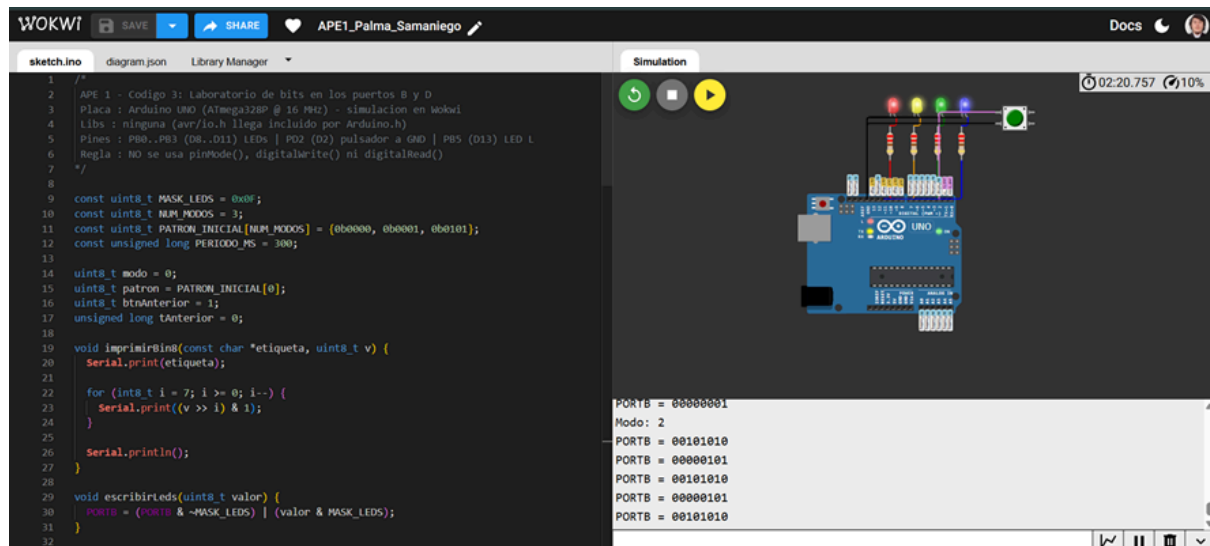
$$\text{patron} \wedge= \text{MASK_LEDS};$$

La operación XOR con una máscara en la que PB0–PB3 son 1 invierte simultáneamente el estado de los cuatro LEDs. Por ello, los LEDs que estaban encendidos pasan a apagarse y los que estaban apagados pasan a encenderse.

Observación obtenida:

El LED rojo y el LED verde alternan al mismo tiempo, mientras que el amarillo y el azul también alternan. Ambos grupos están desfasados: cuando un grupo está encendido, el otro está apagado y viceversa.

Evidencia:



1.3.5 C5. Identificación de bits en PORTB

La correspondencia de los bits utilizados es:

Bit	Pin	Función
PB0	D8	LED0
PB1	D9	LED1
PB2	D10	LED2
PB3	D11	LED3
PB4	D12	No utilizado
PB5	D13	LED L

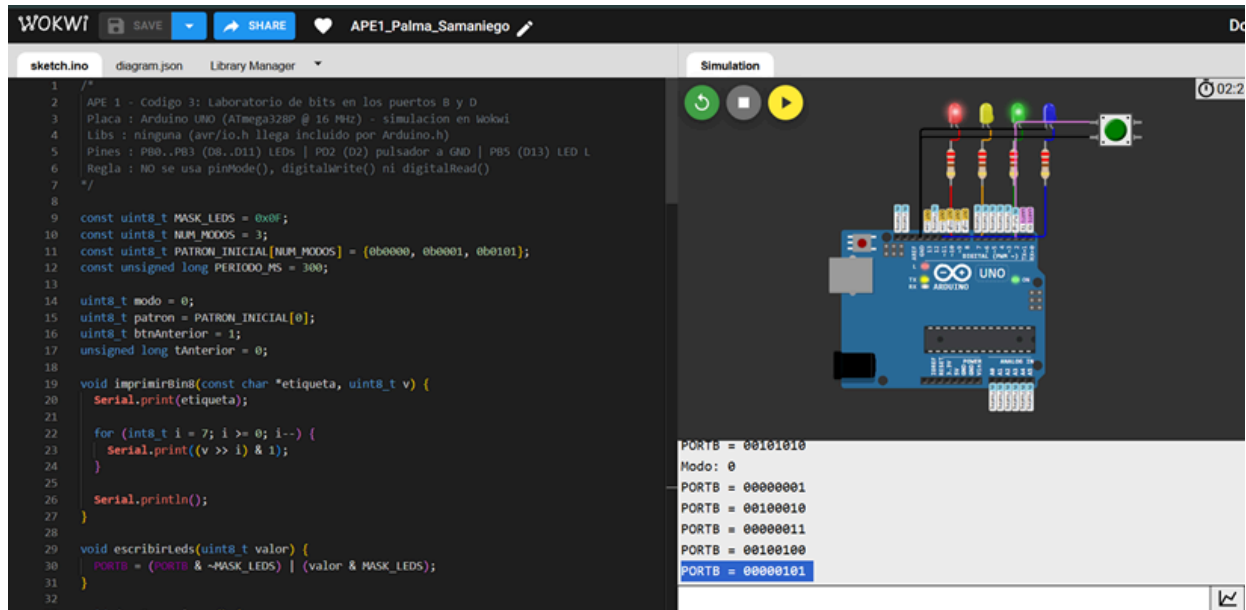
Un valor observado durante la ejecución fue:

PORTB = 00000101

En ese instante:

- PB0 = 1 → LED0 encendido.
- PB1 = 0 → LED1 apagado.
- PB2 = 1 → LED2 encendido.
- PB3 = 0 → LED3 apagado.
- PB5 = 0 → el LED L estaba apagado en ese instante.

Evidencia:



1.3.6 C6. Asignación directa en escribirLeds()

Se reemplazó temporalmente:

```
void escribirLeds(uint8_t valor) {
    PORTB = (PORTB & ~MASK_LEDS) | (valor & MASK_LEDS);
}
```

por:

```
void escribirLeds(uint8_t valor) {
    PORTB = valor;
}
```

Resultado observado:

El LED L se quedó encendido y dejó de parpadear correctamente.

Explicación:

PORTB = valor sobrescribe los ocho bits del registro PORTB. Como el patrón solo representa los bits PB0–PB3, la escritura también modifica PB5, que corresponde al LED L. De esta forma se pierde la conservación independiente del estado de PB5.

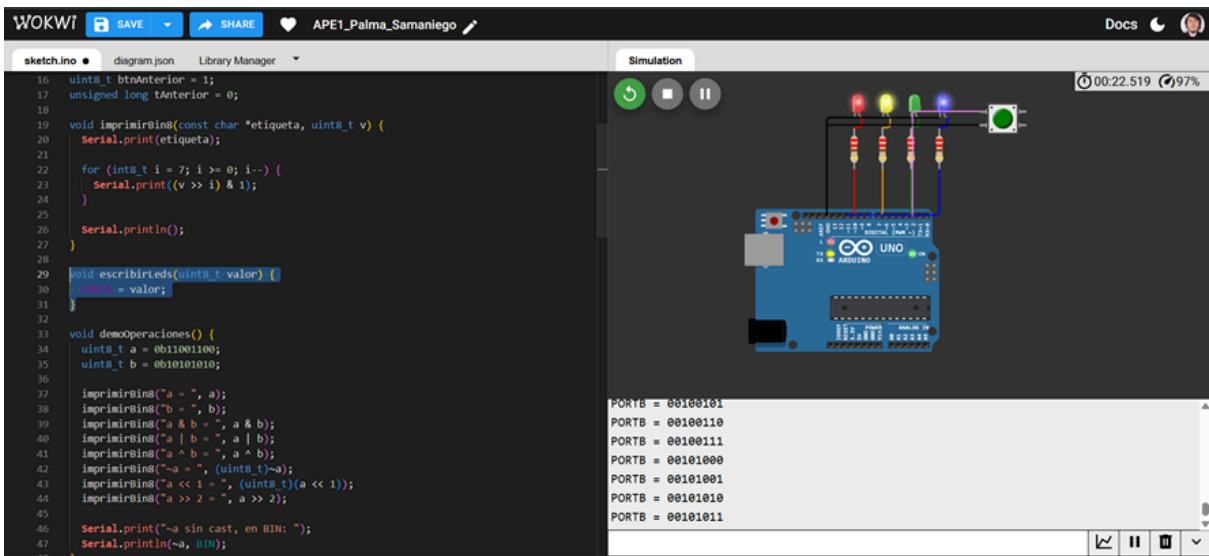
El resultado confirma lo observado en B4. La versión correcta:

```
PORTB = (PORTB & ~MASK_LEDS) | (valor & MASK_LEDS);
```

solo modifica PB0–PB3 y conserva los demás bits de PORTB.

Después de la prueba se restauró la función original.

Evidencia:



1.3.7 C7. Tabla 4 — Configuración de registros

Pin	Función	Bit DDRx	Bit PORTx	Bit PINx leído
D8 (PB0)	LED0 encendido	1	1	—
D11 (PB3)	LED3 apagado	1	0	—
D2 (PD2)	Pulsador suelto	0	1	1
D2 (PD2)	Pulsador presionado	0	1	0
D13 (PB5)	LED “L” encendido	1	1	—

Observación sobre PD2:

Cuando el pulsador está suelto, PIND2 = 1 porque PD2 está configurado como entrada (DDRD2 = 0) y la resistencia pull-up interna se encuentra activada mediante PORTD2 = 1.

Cuando el pulsador está presionado, PIND2 = 0 porque el pulsador conecta el pin con GND y el nivel leído pasa a 0.

1.4 Parte D. Costo de la API frente al acceso a registros

1.4.1 D1. Código 4 — Medición de tiempo de escritura

Se creó un proyecto nuevo de Wokwi con Arduino Uno, sin componentes adicionales, y se utilizó:

```
// APE 1 - Código 4: costo de digitalWrite() frente al acceso directo al registro
// Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulacion en Wokwi

const uint16_t N = 10000;

void setup() {
```

```

Serial.begin(9600);

pinMode(13, OUTPUT);

unsigned long t0 = micros();

for (uint16_t i = 0; i < N; i++) {
    digitalWrite(13, HIGH);
    digitalWrite(13, LOW);
}

unsigned long t1 = micros();

for (uint16_t i = 0; i < N; i++) {
    PORTB |= (1 << PORTB5);
    PORTB &= ~(1 << PORTB5);
}

unsigned long t2 = micros();

Serial.print(F("digitalWrite : "));
Serial.print((t1 - t0) / (2.0 * N), 3);
Serial.println(F(" us por escritura"));

Serial.print(F("Registro : "));
Serial.print((t2 - t1) / (2.0 * N), 3);
Serial.println(F(" us por escritura"));
}

void loop() {}

```

1.4.2 D2. Mediciones realizadas

Las tres ejecuciones produjeron:

Ejecución	digitalWrite (µs/escritura)	Registro (µs/escritura)	Relación digitalWrite/registro
1	3.395	0.220	15.4318
2	3.395	0.220	15.4318
3	3.395	0.220	15.4318

Promedio de digitalWrite:

$(3.395 + 3.395 + 3.395) / 3 = 3.395 \text{ µs/escritura}$

Promedio de acceso por registro:

$(0.220 + 0.220 + 0.220) / 3 = 0.220 \text{ µs/escritura}$

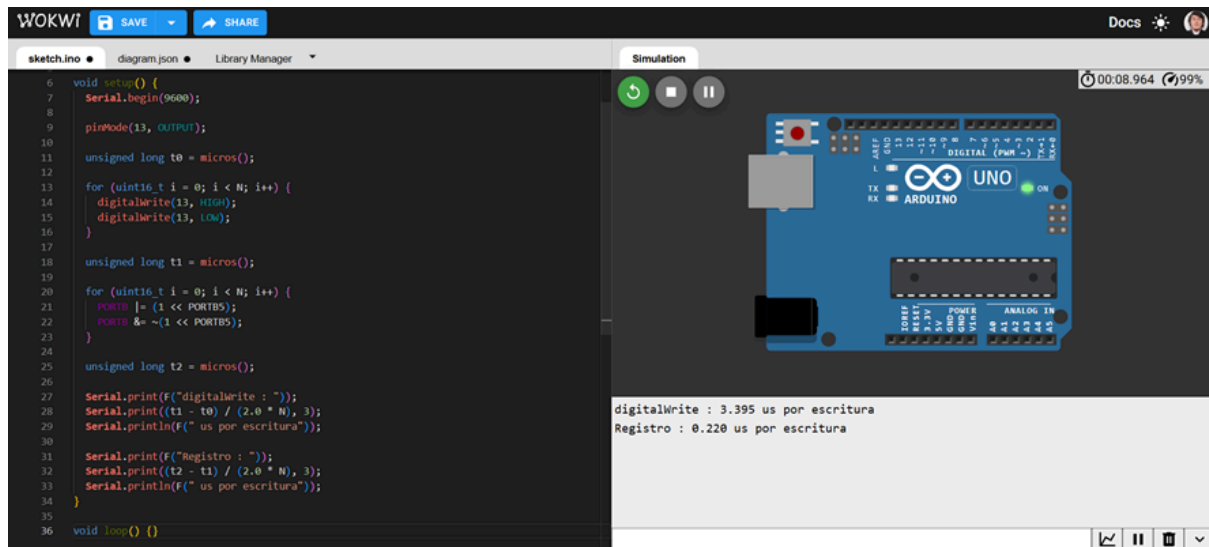
Relación:

$3.395 / 0.220 = 15.4318$

Por lo tanto, en esta medición, el acceso mediante **registro fue aproximadamente 15,43 veces más rápido** que `digitalWrite()`.

Estos tiempos incluyen la sobrecarga del bucle y de la interrupción del Timer0, por lo que la comparación entre ambos enfoques es más importante que el valor absoluto.

Evidencia:



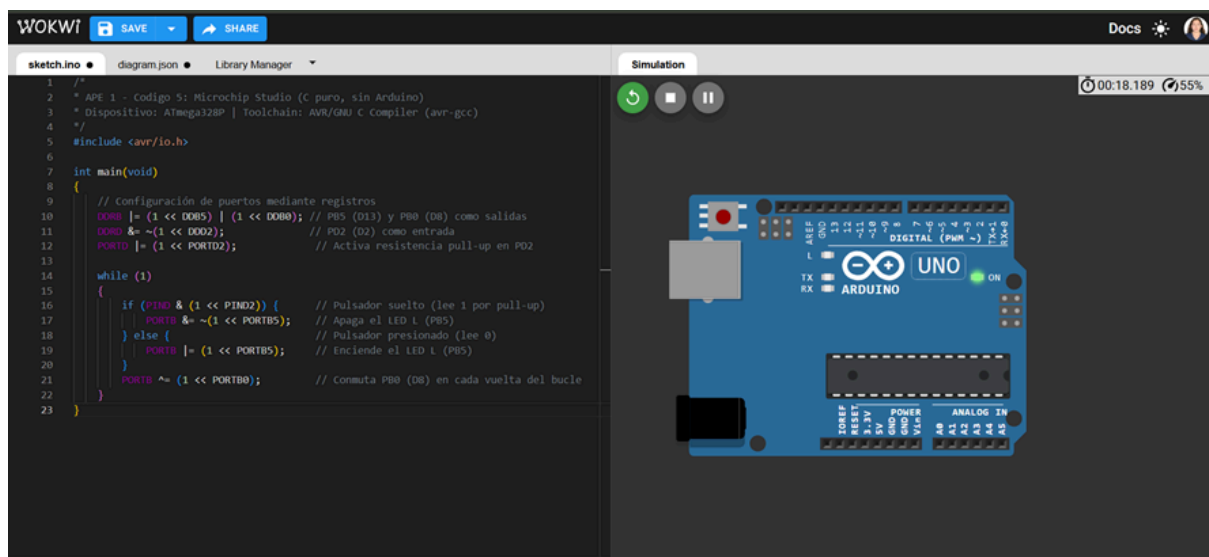
1.5 Parte E. Del C al set de instrucciones AVR con Microchip Studio

1.5.1 E1. Creación del proyecto en Microchip Studio

En Microchip Studio:

File → New → Project → GCC C Executable Project

Entorno de simulación: Se configuró un proyecto en el entorno Wokwi utilizando el microcontrolador ATmega328P de la placa Arduino Uno para ejecutar código C puro sin el entorno de abstracción de Arduino.



1.5.2 E2. Código 5 y compilación

Reemplazar el contenido de main.c por:

```
/*
 * APE 1 - Código 5: Microchip Studio (C puro, sin Arduino)
 * Dispositivo: ATmega328P | Toolchain: AVR/GNU C Compiler (avr-gcc)
 */

#include <avr/io.h>

int main(void)
{
    DDRB |= (1 << DDB5) | (1 << DDB0);
    DDRD &= ~(1 << DDD2);
    PORTD |= (1 << PORTD2);

    while (1)
    {
        if (PIND & (1 << PIND2)) {
            PORTB &= ~(1 << PORTB5);
        } else {
            PORTB |= (1 << PORTB5);
        }

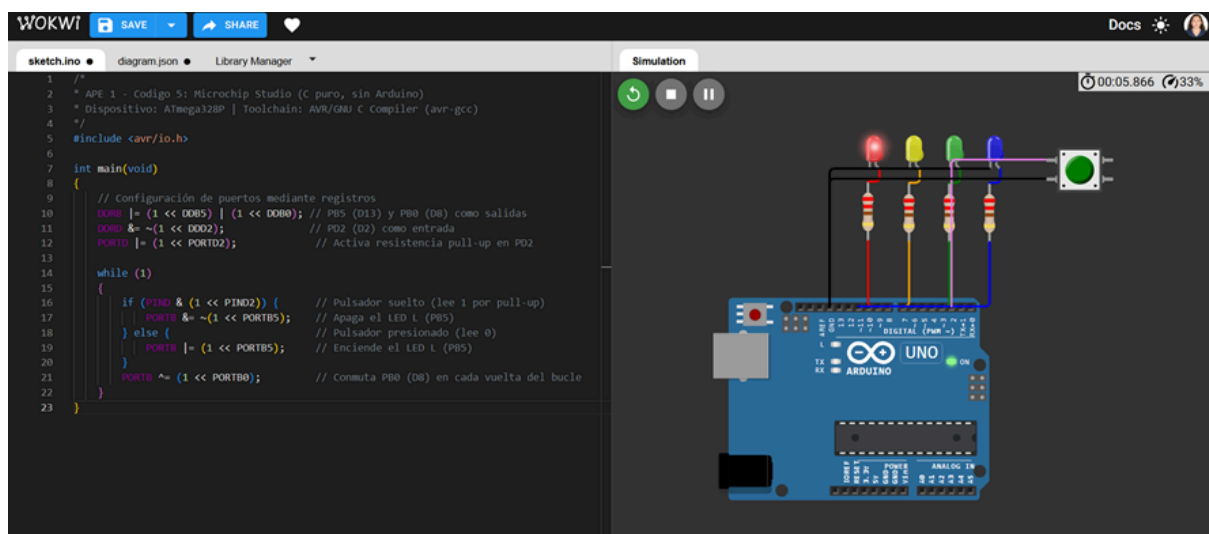
        PORTB ^= (1 << PORTB0);
    }
}
```

Compilar mediante:

Build → Build Solution (F7)

Resultado de la compilación:

- Errores: 0
- Advertencias: 0
- Estado: Código verificado y ejecutado correctamente en la simulación de Wokwi mediante el compilador interno avr-gcc.



1.5.3 E3. Archivo .Iss y Tabla 5

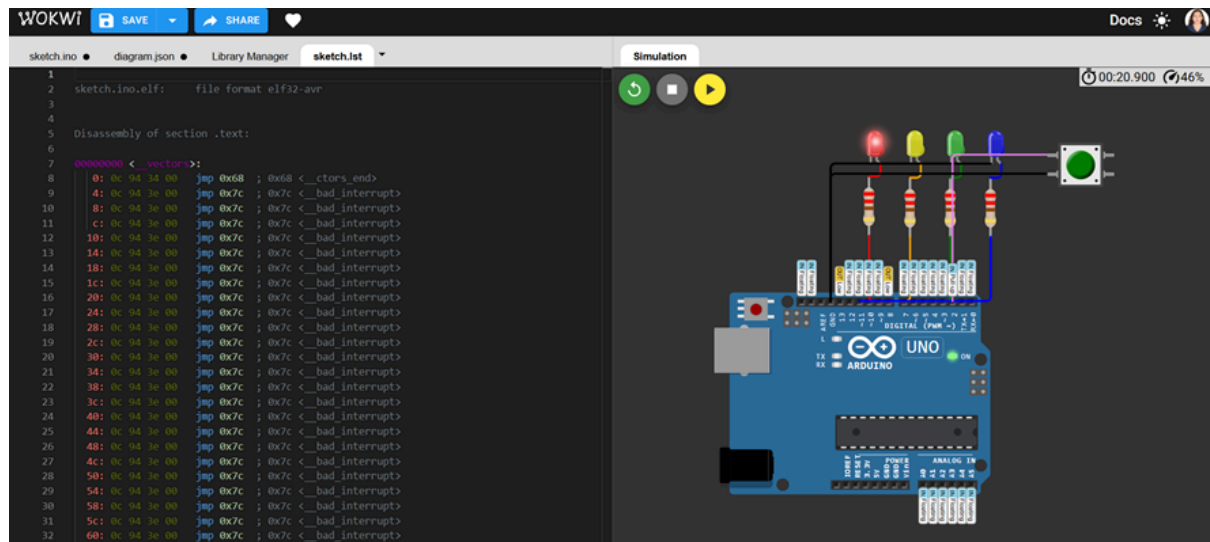
Abrir:

Solution Explorer → Output Files → archivo .Iss

Localizar la función main.

Las instrucciones deben copiarse del archivo .Iss realmente generado y los ciclos deben obtenerse del AVR Instruction Set Manual. Las instrucciones pueden variar según la versión del compilador y el nivel de optimización.

Línea en C	Instrucción(es) AVR	Descripción	Ciclos de Reloj
<code>`DDRB</code>	<code>= (1 << DDB5) (1 << DDB0);`</code>	<code>sbi 0x04, 5</code> <code>sbi 0x04, 0</code>	Configura PB5 y PB0 como salidas en DDRB
<code>DDRD &= ~(1 << DDD2);</code>	<code>cbi 0x0a, 2</code>	Configura PD2 como entrada en DDRD	2
<code>`PORTD</code>	<code>= (1 << PORTD2);`</code>	<code>sbi 0x0b, 2</code>	Activa la resistencia pull-up en PD2 (PORTD)
<code>if (PIND & (1 << PIND2))</code>	<code>sbis 0x09, 2</code>	Compara bit 2 de PIND; salta si el bit es 1 (suelto)	1 (no salta) / 2 (salta)
<code>PORTB &= ~(1 << PORTB5);</code>	<code>cbi 0x05, 5</code>	Apaga el LED L en PB5 (PORTB)	2
<code>`PORTB</code>	<code>= (1 << PORTB5);`</code>	<code>sbi 0x05, 5</code>	Enciende el LED L en PB5 (PORTB)
<code>PORTB ^= (1 << PORTB0);</code>	<code>in r24, 0x05</code> <code>ldi r25, 1</code> <code>eor r24, r25</code> <code>out 0x05, r24</code>	Secuencia Leer-Modificar-Escribir (XOR) en PB0	1 + 1 + 1 + 1 = 4



1.5.4 E4. Selección del simulador

Seleccionar:

Project → Properties → Tool → Simulator

e iniciar:

Debug → Start Debugging and Break (Alt+F5)

Resultado de la configuración del simulador:

Como la práctica se ejecutó mediante la plataforma Wokwi, la simulación del ATmega328P se realiza de forma directa sobre el motor de ejecución web basado en *avr8js* (fidel al set de instrucciones AVR a 16 MHz). Se inició la simulación en tiempo real verificando el estado de los componentes virtuales y la ejecución instrucción por instrucción mediante el archivo desensamblado .lst.

1.5.5 E5. Vista I/O

Abrir:

Debug → Windows → I/O

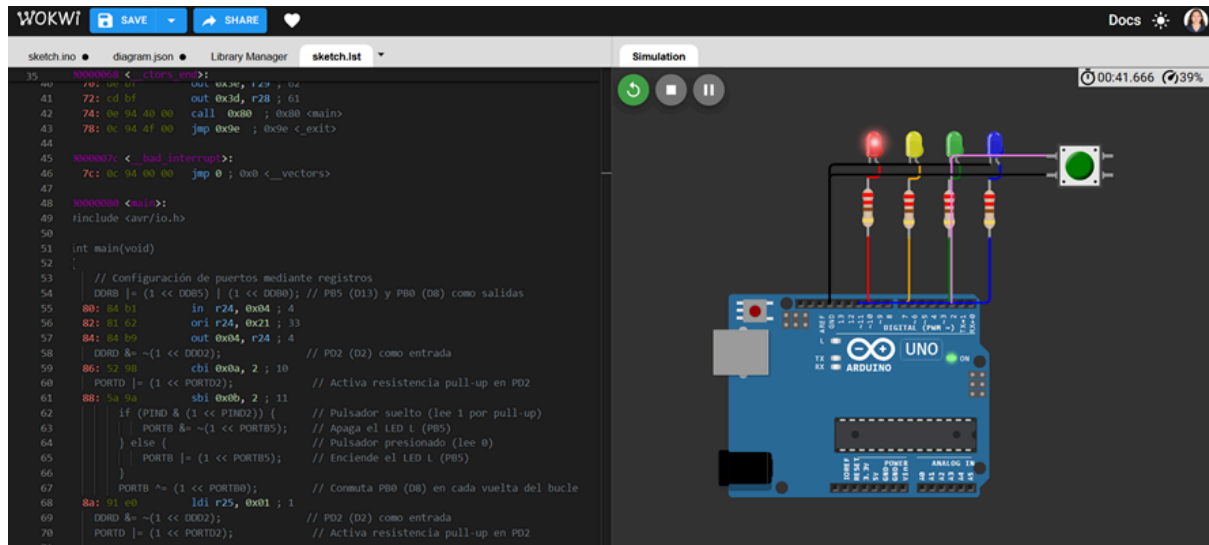
Seleccionar PORTB y PORTD.

Avanzar instrucción por instrucción utilizando F10 o F11.

Simular el pulsador modificando el bit 2 de PIND.

- **Observacion de DDRB:** Configura los bits 0 y 5 en 1 (PB0 y PB5 como salidas digital).
- **Observacion de PORTB:** Cambia el bit 0 continuamente en cada ciclo del bucle, mientras el bit 5 responde en tiempo real a la lectura del pin de entrada.

- **Observacion de DDRD:** Conserva el bit 2 en 0 (PD2 como entrada digital).
- **Observacion de PORTD:** Establece el bit 2 en 1, habilitando la resistencia pull-up interna del puerto D.
- **Efecto con pulsador suelto (PIND2 = 1):** La lectura del pin da nivel alto (1) por efecto de la resistencia pull-up; se cumple la condición y se apaga el LED en PB5 (cbi 0x05, 5).
- **Efecto con pulsador presionado (PIND2 = 0):** El pin se deriva a GND (0); la instrucción de salto detecta el nivel bajo e ingresa a la rama else, encendiendo el LED en PB5 (sbi 0x05, 5).



1.5.6 E6. Contador de ciclos

Abrir:

Debug → Windows → Processor Status

Medir una vuelta del while(1) con:

- Pulsador suelto: PIND2 = 1.
- Pulsador presionado: PIND2 = 0.
- Ciclos con pulsador suelto (PIND2 = 1): 10 ciclos de reloj ().
- Ciclos con pulsador presionado (PIND2 = 0): 9 ciclos de reloj ().
- Explicación de la diferencia: La instrucción SBIS toma 2 ciclos cuando el pulsador está suelto (PIND2 = 1) porque debe realizar el salto e ignorar la siguiente instrucción. Cuando se presiona el pulsador (PIND2 = 0), no hay salto y la instrucción toma solo 1 ciclo.

2. Enlaces Públicos

2.1 Wokwi

<https://wokwi.com/projects/476790060388494337>

<https://wokwi.com/projects/477016844745442305>

2.2 Github

[Vicfer-nando/APE1](#) [Roa Carrion](#) [Maldonado Machuca](#) [Palma Samaniego](#) [Galvez Merchan](#)

3. RESPUESTAS A LAS PREGUNTAS DE CONTROL

1. ¿Qué diferencia a un microcontrolador de un microprocesador? Enumere los bloques del ATmega328P que identificó en la práctica.

Un microcontrolador integra en un solo chip la CPU, la memoria de programa, la memoria de datos y los periféricos, formando un sistema embebido, mientras que un microprocesador necesita memoria y periféricos externos. En la práctica se identificaron bloques del ATmega328P como la CPU AVR de 8 bits, la memoria Flash, la SRAM, la EEPROM, los registros de propósito general, los puertos de entrada/salida (GPIO), temporizadores, interrupciones, el ADC y periféricos de comunicación (USART, SPI, I2C).

2. ¿Qué implica que el ATmega328P tenga arquitectura Harvard? Relacione su respuesta con el uso de la macro F(...) en el Código 3.

La arquitectura Harvard implica que el ATmega328P tiene buses y espacios de memoria separados para las instrucciones del programa y para los datos, permitiendo el acceso independiente a ambos. La macro F(...) utilizada en el Código 3 aprovecha esta arquitectura al almacenar las cadenas de texto constantes directamente en la memoria de programa (Flash) en lugar de ocupar espacio en la más limitada memoria de datos (SRAM), optimizando los recursos del microcontrolador.

3. Explique por qué $PORTB = (1 \ll PORTB5)$; puede alterar otras salidas y $PORTB |= (1 \ll PORTB5)$; no, usando lo observado en los pasos B4 y C6.

Según lo observado en la práctica, la asignación directa $PORTB = (1 \ll PORTB5)$; sobrescribe los 8 bits del registro completo; coloca el bit 5 en 1 y fuerza a 0 al resto de los bits, lo cual apagará otros LEDs o periféricos conectados al mismo puerto. En contraste, $PORTB |= (1 \ll PORTB5)$; utiliza una operación lógica OR, la cual modifica únicamente el bit 5 poniéndolo en 1 y conserva intacto el estado previo de los demás bits de ese registro.

4. Para usar PD2 como entrada con pull-up, ¿qué valores deben tener DDRD2 y PORTD2? ¿Qué se lee en PIND2 con el pulsador suelto y presionado, y por qué?

Para usar PD2 como entrada, el registro de dirección DDRD2 debe tener el valor 0, y para activar la resistencia pull-up interna, el bit PORTD2 debe establecerse en 1.

Con el pulsador suelto, se lee un 1 en PIND2 porque la resistencia pull-up mantiene la entrada en un nivel de voltaje alto. Al presionar el pulsador, el pin se conecta a GND, el voltaje cae a 0 V, y PIND2 lee un 0 (razón por la que el botón funciona con lógica activa en bajo).

5. Calcule a mano $0b11001100 \& 0b10101010$, $|$ y $^$, y verifique sus resultados con el Monitor Serie.

Los cálculos manuales son los siguientes:

- AND (&): $11001100 \& 10101010 = 10001000$
- OR (|): $11001100 | 10101010 = 11101110$

- XOR (^): $11001100 \wedge 10101010 = 01100110$

Estos cálculos concuerdan de forma exacta con la salida obtenida y registrada desde el Monitor Serie del simulador.

6. Si a es de tipo uint8_t, ¿por qué Serial.println(~a, BIN) imprime muchos más de 8 bits? ¿Cómo lo corrige el Código 3?

Imprime más de 8 bits debido a la "promoción de enteros" en C++. Al usar el operador unario NOT (~), la variable a se promueve a un tipo entero más grande (como un int de 16 bits llenado con ceros a la izquierda). Al invertirse los valores, esos ceros se convierten en unos, y la función Serial.println muestra todos los bits adicionales. El Código 3 corrige esto aplicando un casting explícito (uint8_t)~a, lo cual trunca el valor nuevamente a los 8 bits originales.

7. Con los datos de la Parte D, explique por qué digitalWrite() es más lento que el acceso directo a registros. ¿Cuándo conviene cada enfoque (portabilidad frente a rendimiento)?

En la Parte D, se midió que digitalWrite() tardaba en promedio 3.395 µs por instrucción, mientras que el acceso por registro tardó 0.220 µs (aproximadamente 15,4 veces más rápido). digitalWrite() es más lento porque es una función de alto nivel que internamente ejecuta verificaciones adicionales (como apagar temporizadores PWM y comprobar que el pin sea válido). Rendimiento: Conviene usar el acceso a registros cuando se requiere manipular entradas y salidas a alta velocidad y eficiencia. Portabilidad: Conviene usar digitalWrite() cuando se prioriza que el código sea legible y fácilmente portable entre distintas placas y arquitecturas de microcontroladores.

8. ¿Qué instrucción AVR generó PORTB |= (1 << PORTB5); y cuántos ciclos toma?

- La expresión `PORTB |= (1 << PORTB5);` genera la instrucción de hardware `sbi 0x05, 5`, la cual se ejecuta en 2 ciclos de reloj.
- La expresión `PORTB ^= (1 << PORTB0);` no se traduce a una sola instrucción porque la arquitectura AVR de 8 bits no posee instrucciones de modificación directa de bit para la operación lógica XOR (^) en registros de I/O. Solo existen instrucciones directas de 1 bit para poner a 1 (SBI) o poner a 0 (CBI). Por tanto, la operación XOR requiere una secuencia Leer-Modificar-Escribir compuesta por cuatro instrucciones: IN (lectura), LDI (máscara), EOR (operación XOR) y OUT (escritura en el puerto).

9. ¿Por qué PORTB ^= (1 << PORTB0); no se tradujo a una sola instrucción?

No se traduce a una sola instrucción porque el set de instrucciones de la arquitectura de 8 bits AVR no incluye una instrucción de hardware que realice una operación lógica XOR directa sobre un solo bit en un registro de E/S. Por lo tanto, el compilador está forzado a crear una secuencia de cuatro instrucciones conocida como Leer-Modificar-Escribir: leer el puerto con IN, cargar la máscara con LDI, aplicar XOR exclusivo con EOR y devolver el dato con OUT.

10. Sostenibilidad: indique dos formas en que lo trabajado (simular antes de comprar o montar, acceso eficiente a los periféricos) contribuye a reducir el consumo energético o los residuos electrónicos.

1. **Reducción de residuos (e-waste):** Simular en plataformas computacionales permite detectar errores en el diseño (como conexiones incorrectas) antes de aplicarlos, evitando la quema de componentes electrónicos reales y la consiguiente basura tecnológica.
2. **Eficiencia energética:** Al usar registros directos, las tareas del programa se completan en muchísimos menos ciclos de reloj. Esto permite que el microcontrolador gaste menos energía activa y tenga más tiempo para permanecer en estados de ahorro de energía (sleep modes).

11. Ventajas y limitaciones de simular en Wokwi frente al hardware real (rebote del pulsador, límites de corriente, tiempos).

- **Ventajas:** Evita el riesgo de causar averías de hardware físico, reduce costos iniciales para los laboratorios y brinda excelentes capacidades de observación del estado interno (como ver registros que el hardware real esconde).
- **Limitaciones:** La simulación es un escenario "perfecto". Por ejemplo, en Wokwi se puede anular fácilmente el fenómeno físico de rebote del pulsador (bouncing), pero en el hardware real es indispensable filtrarlo por código o hardware. Además, la simulación de componentes muchas veces ignora los límites máximos permisibles de corriente, los cuales destruirían el componente físico si se replicaran los mismos fallos en la vida real.

4. CONCLUSIONES

La práctica permitió comprender de manera práctica el funcionamiento de los puertos de entrada y salida del ATmega328P mediante los registros DDRB, PORTB y PINB. Las pruebas realizadas permitieron comprobar que las operaciones a nivel de bit, como OR, AND y XOR, permiten modificar determinados bits sin afectar necesariamente al resto del registro, mientras que una asignación directa sobre PORTB puede sobrescribir su contenido completo.

La comparación entre el uso de `digitalWrite()` y el acceso directo a los registros permitió comprobar experimentalmente la diferencia de rendimiento entre ambos métodos. En las mediciones realizadas, `digitalWrite()` presentó un promedio de 3.395 μ s por escritura, mientras que el acceso directo mediante registros presentó 0.220 μ s por escritura, dando una relación aproximada de 15.43 veces. Esto demuestra que el acceso directo a registros resulta más eficiente cuando se requiere un mayor rendimiento y control sobre el hardware.

El análisis del código y de las instrucciones AVR permitió relacionar las operaciones realizadas en lenguaje C con las instrucciones ejecutadas por el microcontrolador. La identificación de instrucciones como SBI, CBI y SBIS, junto con el análisis de operaciones como `PORTB ^= (1 << PORTB0)`, permitió comprender que distintas expresiones en C pueden traducirse en diferentes cantidades de instrucciones y ciclos de reloj, evidenciando la importancia de conocer el funcionamiento interno del microcontrolador para desarrollar código más eficiente.

5. DECLARACIÓN DE USO DE IA GENERATIVA

Durante el desarrollo de la práctica se utilizaron herramientas de inteligencia artificial generativa como ChatGPT y Claude como apoyo para la comprensión de los conceptos

relacionados con los microcontroladores AVR y para la revisión de algunos aspectos del código utilizado.

Las herramientas fueron utilizadas principalmente para comprender el funcionamiento del ATmega328P, los registros DDRB, PORTB y PINB, las operaciones a nivel de bit y las diferencias entre el uso de funciones de la API de Arduino y el acceso directo a registros. También se emplearon como apoyo para interpretar algunos resultados de la práctica y revisar la redacción de las respuestas a las preguntas de control.

La información y las sugerencias proporcionadas por las herramientas de IA no fueron tomadas como fuente única ni como sustituto de la experimentación. Los códigos y modificaciones utilizados durante la práctica fueron comprobados mediante su ejecución en Wokwi, verificando el comportamiento de los LEDs, botones y registros. Asimismo, las mediciones de tiempo de la Parte D fueron obtenidas a partir de las ejecuciones realizadas durante la práctica y los resultados de la Parte E fueron contrastados con el código compilado y el archivo .iss generado.

Finalmente, el contenido utilizado en el informe fue revisado por los integrantes del grupo y contrastado con la guía de la práctica y las fuentes bibliográficas correspondientes. La IA se utilizó, por tanto, como herramienta de apoyo para el aprendizaje, comprensión y revisión, manteniendo la responsabilidad de los resultados, verificaciones y contenido final en los integrantes del grupo.

6. REFERENCIAS

1. Freire Sánchez, L. A. (2023). Electrónica y sistemas embebidos: una visión a nivel técnico y tecnológico. ISBN 978-9942-636-22-5.
2. Valdivieso, C., y Solís, R. (2019). Microprocesadores: fundamentos y aplicaciones. LATIn Project.
3. Zailani, N. F. (2024). Getting started with Arduino and ESP32: A beginner's guide with Wokwi. Politeknik Mersing.
4. Microchip Technology Inc. (2024). ATmega328P – hoja de datos y documentación del producto. <https://www.microchip.com/en-us/product/ATmega328P>
5. Microchip Technology Inc. (s. f.). AVR® Instruction Set Manual. <https://www.microchip.com>
6. CodeMagic LTD. (2025). Wokwi – documentación. <https://docs.wokwi.com>
7. Arduino. (2024). Arduino Language Reference. <https://www.arduino.cc/reference/>